Grover's Algorithm

QC@IITI

Introduction

Grover's Algorithm is built to solve the unstructured search problems and offers a quadratic imporvement over the classical search algorithms. Within an unstructured database, where all elements are randomly placed, the best possible way to search through it is to just check all elements one by one.

If we want to find a *'Key'*, or a certain element within the unstructured database by the classical way we can say that the time complexity of classical search is given by $O(N)$ (Explanation attached further within the text). Now, Grover's Algorithm, on the other hand, is able to find the marked element, or the 'Key' with a time complexity of $O(\sqrt{N})$. It does so by using two "Quantum Oracles", they being the Grover's Oracle and the Diffusion Oracle. In this text, we will learn what these oracles are, what are they made of, how are they used, and how they affect a state to provide speedup for key searching.

The Algorithm

The Classical Way

Say, we want to find a *'Key'*, or a certain element within the unstructured database. By following the classical method, best case scenario is, for *'N'* elements within the database, the first element you search happens to be the *Key*, or the element which you are searching for. Worst case scenario is that, if you check 'N - 1' elements, and still have not found the key, you know for a fact that the key is the last element within the database. This can be compared to blindly searching within a database randomly, which it effectively is. Due to this, we can say that the average number of elements we need to check is $\frac{N}{2}$. Therefore, we say that the Time complexity of the classical search is $O(N)$.

The Quantum Way

Now, we shall understand Grover's Algorithm. First, we define a function f(x) such that $f : \Sigma^n \rightarrow \Sigma$, and $f(x) = 1$ whenever x is the key. This means that, while finding the key, what Grover's Algorithm is doing, is searching for a string $x \epsilon \Sigma^n$ for which $f(x) = 1$.

One more thing to note here, is that 'N' refers to the total number of strings within the database, while *'n'* refers to the number of classical bits or qubits needed to represent it. Now, we will formally define the problem statement which the base Grover's Algorithm solves. We will be naming this problem the 'Unique Search'

Unique Search

Input: a function of the form $f : \Sigma^n \rightarrow \Sigma$

Promise: there is exactly one key string $z \epsilon \Sigma^n$ for which f(z) = 1, with f(x) = 0 for all string $x \neq z$

Output: the string z

Now, to actually implement the algorithm, a few things must first be defined.

Phase Query Gates

Here, to implement Grover's Algorithm, we will first define two phase query gates, $Z_f |x\rangle$ and $Z_{\text{OR}} |x\rangle$.

We will define the first phase query gate $Z_f |x\rangle$ as follows:

$$Z_f |x\rangle = (-1)^{f(x)} |x\rangle$$

The operation Z_f can be implemented using one query gate U_f. An example for a system of 11 states is visually attached:

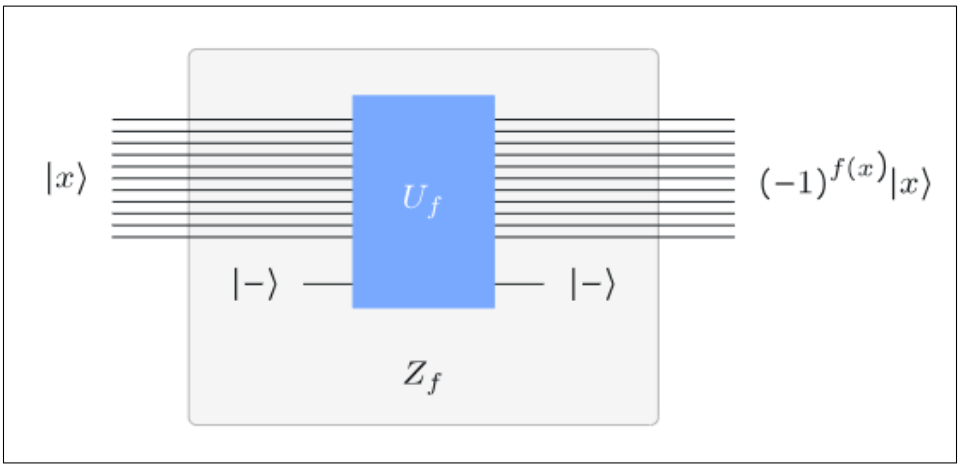

Figure 1: Grover's Oracle

We will henceforth refer to this operator as the *Grover's Oracle*. While it might seem complicated either mathematically or even visually, it's purpose is quite simple. All it does, is that for all of the strings $|x\rangle$ that is inputed into it, it only changes the phase of the KeyString by 180°, while leaving all other strings unaffected. The purpose for this will become clear in conjunction with the next operator.

Now, we shall look at the second phase query gate $Z_{OR} |x\rangle$. We will define it as follows:

$$Z_{OR} |x\rangle = \begin{cases} |x\rangle & \text{if } x = 0^n \\ -|x\rangle & \text{if } x \neq 0^n \end{cases}$$

This query gate, in its simplest state, does the singular job of flipping the sign of the *All-Zero State*, while keeping all of the other terms the same. We do not use this query gate in its raw form, but rather, we attach gates before it an after it to form the *Diffusion Oracle*.

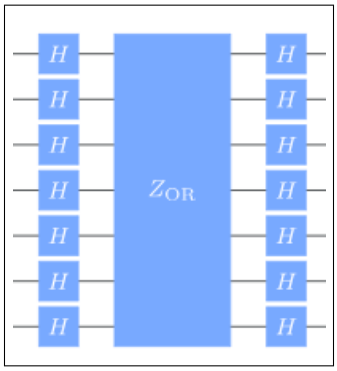

Figure 2: Diffusion Oracle

As you can see here, we have *'Sandwiched'* the Z_{OR} operator between two columns of Hadamard Operators, one on each side. The logic for doing this is quite simple to understand.

As might be known to the reader previously, applying a Hadamard gate commonly to multiple qubits puts the system into a state of equal superposition. In this case, by applying a common Hadamard gate before applying the Z_{OR} gate makes it so that instead of reflecting about the all-zero state, the gate now reflects all the states about their mean state. We can try to understand this mathematically by the following way:

Lets consider the state $|\psi\rangle$ as the state produced when the first row of Hadamard gates have been applied. We can say the following:

$$|\psi\rangle = H^{\otimes n} |0\rangle$$

This is the uniform superposition state, where all possible states have the exact same probability of occuring if measured at this point. Upon this, if we apply the Z_{OR} Query gate, followed by another stack of Hadamard Gates, the resultant is what we will be referring to as the *Diffusion Operator*. Mathematically, we define it as the following:

$$\text{Diffusion Oracle} = 2 |\psi\rangle \langle\psi| - I = H^{\otimes n} Z_{OR} H^{\otimes n} |\psi\rangle$$

This equation is derived mathematically from the operands involved in the entire operation. The proper derivation is not included.

Together, after connecting the Grover's Oracle to the Diffusion Oracle, we now have the main Grover's Operation that will be used throughout the Algorithm. The basic building block of the circuit can be depicted as follows:

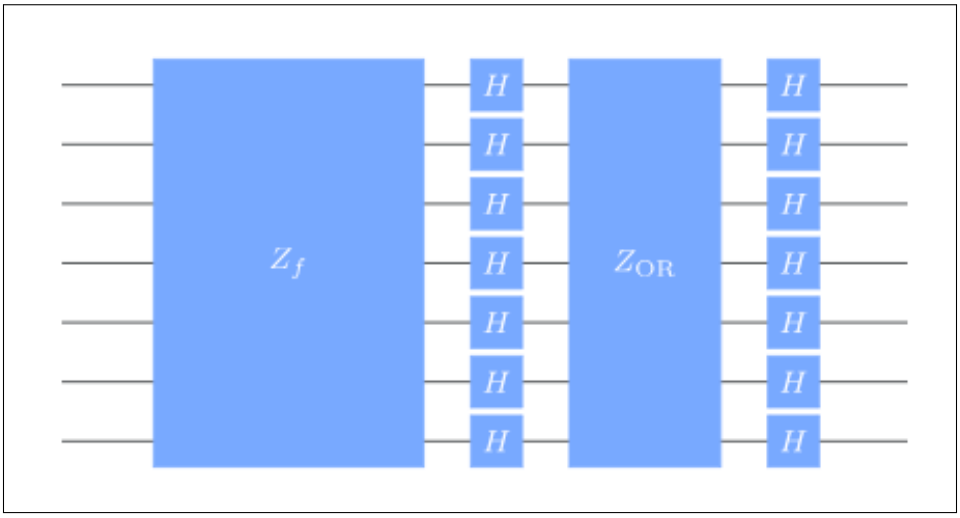

Figure 3: Grover's Operation

Knowing the Gates and what they do mathematically is great, but it is even more important to have an intuition as to what is happening physically within the system. We will now try to build that intuition.

Physical Understanding

Here, it is presumed that the reader knows basics such as the Bloch Sphere and geomertric reflections. Let us define two sets of strings, A_0 and A_1, such that

$$A_0 = \{x \, \epsilon \, \Sigma^n : f(x) = 0\}$$

$$A_1 = \{x \, \epsilon \, \Sigma^n : f(x) = 1\}$$

A_1 contains all of the solutions to the search problem, while A_0 contains the string which arent a solution. Let us define a 2-D space, where the Horizontal Axis depicts A_0 and the vertical axis depicts A_1, now we

can depict any certain state upon this space, since we can define any state $|\psi\rangle$ as the following:

$$|\psi\rangle = C_0 |A_0\rangle + C_1 |A_1\rangle, \textit{where } C_0 \textit{ and } C_1 \textit{ are constants such that } A_0^2 + A_1^2 = 1$$

Now, let us depict the hypothetical unit vector $|\psi\rangle$ on the space

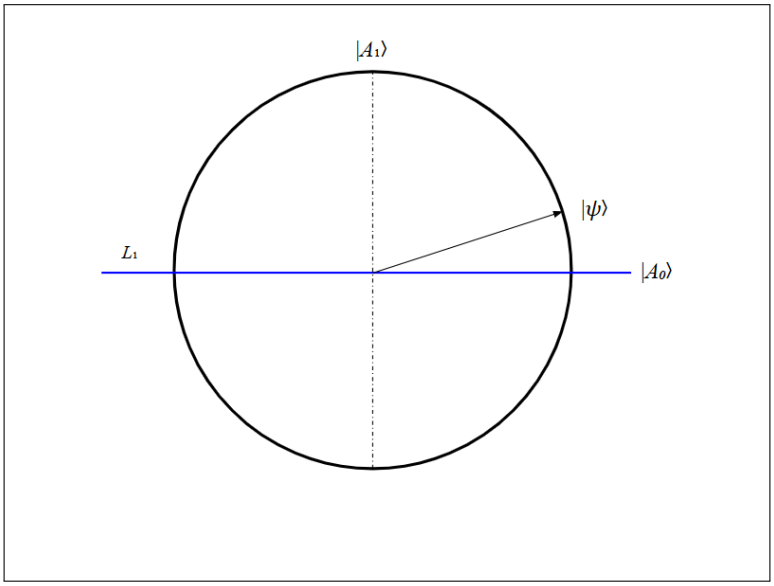

Figure 4: Custom Space

As mentioned earlier, upon applying the Z_f operator, or the Grover's Oracle, the part of $|\psi\rangle$ which contains the Key is phase flipped, which in this case can be considered as having been flipped about L_1. Upon doing this, we obtain the state $Z_f |\psi\rangle$. We can represent the effect of applying Grover's Oracle on the graph the following way:

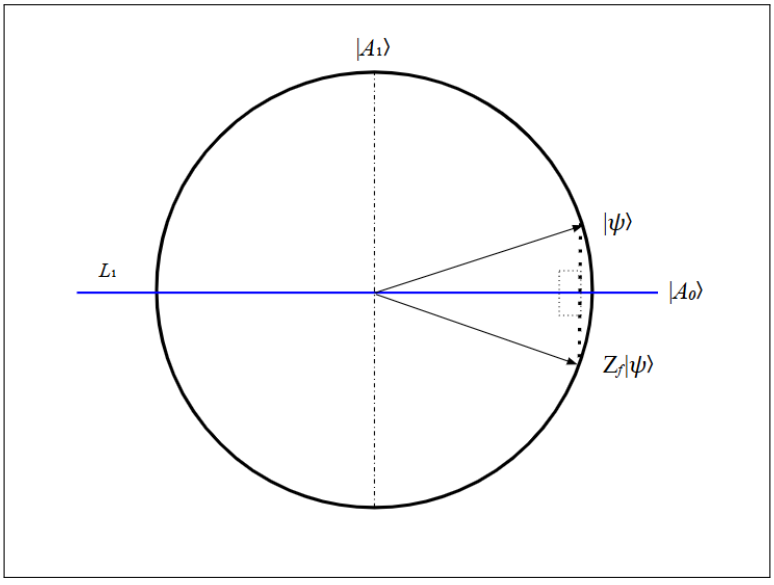

Figure 5: $Z_f |\psi\rangle$ represented in the Custom Space

Next, we must now apply the Diffusion Oracle upon this system. As the reader might recall, we know that the Diffusion Oracle flips the phase of the system about the mean value of the System.

Here, we shall depict the mean value of the system by the line L_2, passing thru the center and vector $|u\rangle$, where $|u\rangle$ is the mean value of the system. The resultant is labelled as $H^{\otimes n} Z_{OR} H^{\otimes n} |\psi\rangle$

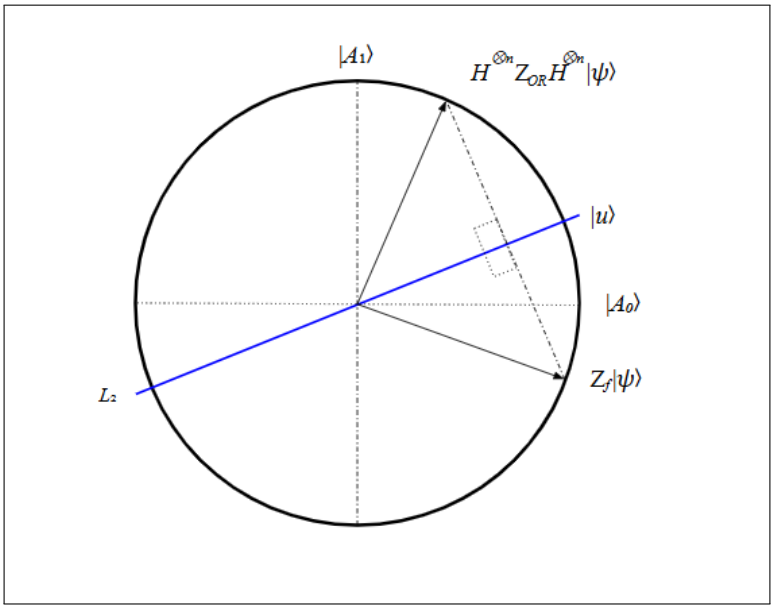

Figure 6: $H^{\otimes n} Z_{OR} H^{\otimes n} |\psi\rangle$ represented in the Custom Space

Putting these two together, we now have produced a net rotation of 2θ degrees from the initial placement of $|\psi\rangle$ in a single application of the Net Operation. Thru further calculation (not attached herewithin) within the mathematical picture, we can find that the value of theta can be given as follows:

$$\theta = \cos^{-1}\left(\sqrt{\frac{|A_0|}{N}}\right)$$

The superimposed image is produced as follows:

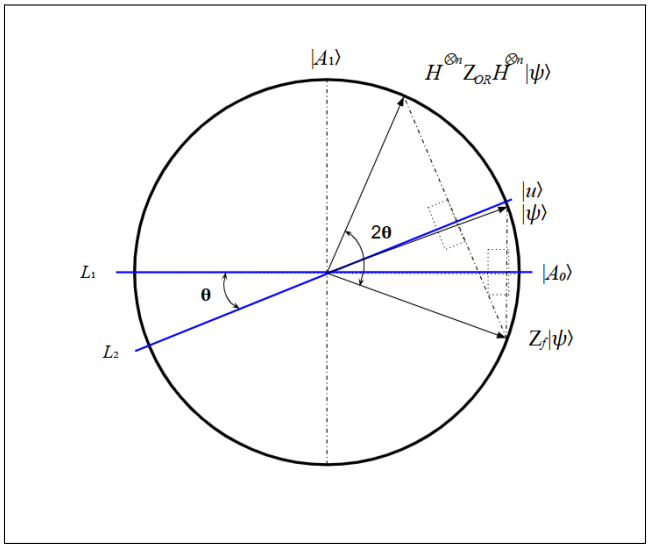

Figure 7: $H^{\otimes n} Z_{OR} H^{\otimes n} |\psi\rangle$ represented in the Custom Space

Iterations

Now that we know what effect a single iteration of the two Operations have upon the state, we must now question ourselves *What Next?*. The answer lies in what we were doing with all the reflections in the first place. The idea behind performing the reflections was to have the arbitrary set of states $|\psi\rangle$ cull away the non-key states represented by $|A_0\rangle$, while increasing the amplitude of the key-states represented by $|A_1\rangle$, which in this case is present on the vertical axis.

So logically, what we are working toward is to make sure that after performing a certain number of Grover Iterations, say 't', the state lies either *ON* $|A_1\rangle$, or *As Close to* $|A_1\rangle$ as possible, since this will lead to the highest probability of the Key-State being read when the state is read. Doing this is quite simple. We just need to choose a 't' which satisfies the following:

$$G^t |u\rangle = cos((2t+1)\theta) |A_0\rangle + sin((2t+1)\theta) |A_1\rangle$$

Here, since we want the state to be as close to vertical as possible, we want the horizontal term to dissapear. So, we must choose t such that:

$$\langle A_1| G^t |u\rangle = sin((2t+1)\theta) = 1 \ , or, \ (2t+1) \approx \frac{\pi}{2}$$

Of course, taking into consideration the fact that t can only be an integer, and that it wont be exact, we can say the following:

$$t \approx \left\lfloor \frac{\pi}{4\theta} \right\rfloor$$

Here, we ignore the $\frac{1}{2}$ since we are only looking for the integer solutions of t. Now, as we might previously remember, we found that $\theta = \cos^{-1}\left(\sqrt{\frac{|A_0|}{N}}\right)$. We can rewrite it to say that, in this verticality case, $\theta = \sin^{-1}\left(\sqrt{\frac{1}{N}}\right)$

As the count of N grows large, we can apply the approximation $\theta = \sin^{-1}\left(\sqrt{\frac{1}{N}}\right) \approx \sqrt{\frac{1}{N}}$. So, finally we get the value for 't' as follows:

$$t = \left\lfloor \frac{4\theta}{\pi}\sqrt{N} \right\rfloor$$

With this, we have now arrived at the end of Grover's Algorithm. We know what Grover's Algorithm solves, what Gates it uses to solve them, what it does to the system both mathematically and graphically, and how many times it is run to produce the solution. All that is left is to understand how we apply it in code, which is attached in the next section.

Code implementation

Grover's Oracle

The Grover's Oracle, given a state, flips the phase of the key-states, in this case, the singular key-state, while making sure the others stay the same.

```
def grover_oracle(qc, num_qubits, target_state):
    controls = list(range(num_qubits - 1))
    target_q = num_qubits - 1
    for i, bit in enumerate(reversed(target_state)):
        if bit == '0':
            qc.x(i)
    qc.h(target_q)
    qc.mcx(controls, target_q)
    qc.h(target_q)
    for i, bit in enumerate(reversed(target_state)):
        if bit == '0':
            qc.x(i)

```

Figure 8: Grover's Oracle

Diffusion Oracle

The Diffusion Oracle flips the states about the mean value. In this case, we have implemented it using a Multi-Controlled Z gate, which is further implemented by Sandwiching a Multi-Controlled X gate within a layer of Hadamards. This was further wrapped between anoter layer of X and Hadamard gates.

```
def diffusion_oracle(qc, num_qubits):
    controls = list(range(num_qubits - 1))
    target_q = num_qubits - 1
    qc.h(range(num_qubits))
    qc.x(range(num_qubits))
    qc.h(target_q)
    qc.mcx(controls, target_q)
    qc.h(target_q)
    qc.x(range(num_qubits))
    qc.h(range(num_qubits))

```

Figure 9: Diffusion Oracle

Running the Code

After deciding the number of qubits, and choosing a random key to be found, we implement the number of iterations 't', before finally applying the two oracles equal to the amount of iterations, before reading the output.

```
num_qubits = 12
target_int = random.randint(0, 2**num_qubits - 1)
target_state = format(target_int, f'0{num_qubits}b')
print(f"Target State: {target_state}")

iterations = math.floor(math.pi / 4 * math.sqrt(2**num_qubits))

qc = QuantumCircuit(num_qubits, num_qubits)
qc.h(range(num_qubits))

for _ in range(iterations):
    grover_oracle(qc, num_qubits, target_state)
    diffusion_oracle(qc, num_qubits)

qc.measure(range(num_qubits), range(num_qubits))

simulator = AerSimulator()
compiled_circuit = transpile(qc, simulator)
result = simulator.run(compiled_circuit, shots=10**6).result()
counts = result.get_counts()

plot_histogram(counts)
plt.show()
```

Figure 10: Running process

Results

The produced output was read for the amount of shots run (in this case, 10^6) and a plot was formed. The formed plot shows how efficient Grover's Algorithm is at finding the key-state. In this particular run, it found the key-state with a 99.9943% probability, which is effectively certain of finding the correct key-state.

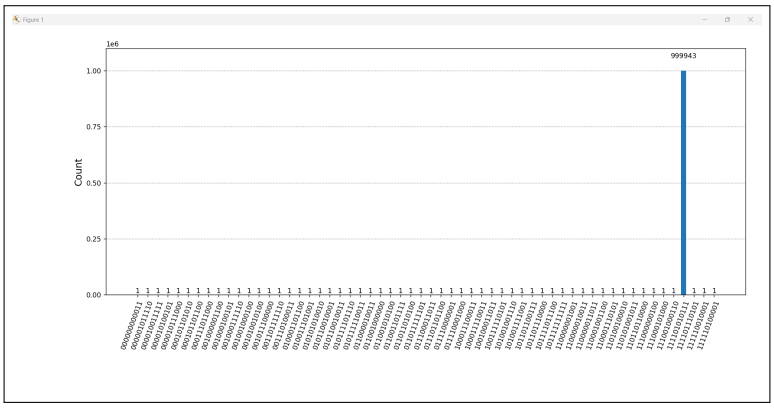

Figure 11: Resultant outputs

Code

Lastly, the code is attached for the ease of the user in case of running it for yourself, or if required in case of testing purposes.

Listing 1: Grover's Algorithm Implementation with Qiskit

```
import math
import random
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

def grover_oracle(qc, num_qubits, target_state):
    controls = list(range(num_qubits - 1))
    target_q = num_qubits - 1
    for i, bit in enumerate(reversed(target_state)):
        if bit == '0':
            qc.x(i)
    qc.h(target_q)
    qc.mcx(controls, target_q)
    qc.h(target_q)
    for i, bit in enumerate(reversed(target_state)):
        if bit == '0':
            qc.x(i)

def diffusion_oracle(qc, num_qubits):
    controls = list(range(num_qubits - 1))
    target_q = num_qubits - 1
    qc.h(range(num_qubits))
    qc.x(range(num_qubits))
    qc.h(target_q)
    qc.mcx(controls, target_q)
    qc.h(target_q)
    qc.x(range(num_qubits))
    qc.h(range(num_qubits))

num_qubits = 12
target_int = random.randint(0, 2**num_qubits - 1)
target_state = format(target_int, f'0{num_qubits}b')
print(f"Target␣State:␣{target_state}")

iterations = math.floor(math.pi / 4 * math.sqrt(2**num_qubits))

qc = QuantumCircuit(num_qubits, num_qubits)
qc.h(range(num_qubits))

for _ in range(iterations):
    grover_oracle(qc, num_qubits, target_state)
    diffusion_oracle(qc, num_qubits)

qc.measure(range(num_qubits), range(num_qubits))

simulator = AerSimulator()
compiled_circuit = transpile(qc, simulator)
result = simulator.run(compiled_circuit, shots=10**6).result()
counts = result.get_counts()

plot_histogram(counts)
plt.show()
```

External Links

Lastly, for further study and query resolution, I have attached a few sites which the reader may refer to:

- In case, for further Physical Understanding on Grover's Algorithm: 3Blue1Brown's Videos: Video 1, Video 2
- For Mathematical understanding: IBM Quantum Learning
- Alternate Code from Xanadu for the Pennylane enjoyers: Pennylane demos